

Basket Management

Basket management provides a consistent way to preserve cart state across the server and client while removing the need to fetch or create a basket on every request. Our React Router middleware stores a snapshot of the current basket as a cookie, enabling the basket ID and item count to persist across page refreshes. Full basket state hydration is performed lazily, which helps keep page loads fast.

Contents

Dependencies

[Overview](#)[Basket Middleware](#)[Basket Provider and Client Hooks](#)[Advanced Configuration](#)[Basket Lifecycle Summary](#)

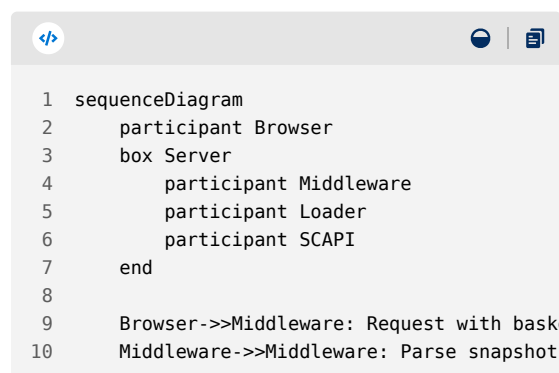
Dependencies

Package	Version	Purpose
React Router 7	7.12.0	Middleware and context system.
React Context	Built-in	Basket provider state.

Overview

Here's an overview of the basket management process.

1. Middleware initializes context with a cookie snapshot on every request.
2. Root loader passes the snapshot into the `BasketProvider` without hydrating the full basket.
3. Client components hydrate lazily via `useBasket()` when full basket data is required.
4. Middleware updates or deletes the cookie based on the final basket state after loaders or actions finish.





```
16 Loader-->>Browser: Return basket snapshot  
17 Middleware->>Middleware: Update cookie state
```

```
</> | |  
  
1 sequenceDiagram  
2     participant Browser  
3     box Client  
4         participant Provider  
5     end  
6     participant SCAPI  
7  
8     Browser->>Provider: useBasket() called  
9     Provider->>SCAPI: getOrCreateBasket(basketId)  
10    SCAPI-->>Provider: Full basket payload  
11    Provider-->>Browser: Context hydrated
```

Basket Middleware

The server-side middleware is the source of truth for basket state and cookie management.

Configuration Options

`createBasketMiddleware` accepts these optional settings.

- `mode`: 'lazy' (default) or 'eager' hydration.
- `cookieName`: Override the basket cookie name (default `__sfcd_basket`).
- `cookieDurationRegistered`: Cookie duration (TTL) in ms for registered shoppers.
- `cookieDurationGuest`: Cookie duration (TTL) in ms for guest shoppers.
- `currency`: Currency used when creating a basket.
- `calculateBasketSnapshot`: Add custom fields to the cookie snapshot.

Note

The cookie duration (TTL) default values are set to the same default values for your ECOM instance. If you have customized those values, you'll want to ensure that you are properly configuring these values to be in alignment.

Context value:

- `basketResourceContext`: snapshot, basket, hydration state, and error.

Modes:



Cookie snapshot characteristics: The cookie snapshot:

- Is stored in `__sfdc_basket`.
- Includes `basketId`, `totalItemCount`, and `uniqueProductCount`.
- Contains optional custom fields via `calculateBasketSnapshot`.

Here's a minimal cookie snapshot configuration for the default lazy mode.

```
1 // root.tsx
2 import createBasketMiddleware from "@middle
3
4 export const middleware = [
5   // ...other middleware
6   createBasketMiddleware(),
7 ];
```

Context Getters and Mutations

Use these helpers in loaders or actions (server) to interact with the request-scoped basket.

```
1 import {
2   getBasket,
3   getBasketSnapshot,
4   ensureBasketId,
5   updateBasketResource,
6   destroyBasket,
7 } from "@middlewares/basket.server";
8
9 export async function loader({ context }) {
10   const snapshot = getBasketSnapshot(context);
11   const basketResource = await getBasket(context);
12   return { snapshot, basket: basketResource };
13 }
14
15 export async function action({ context }) {
16   const basketId = await ensureBasketId(context);
17   // ...perform SCAPI calls to mutate basket
18   updateBasketResource(context, { current } => {
19     ...current,
20     basketId: basketId ?? current?.basketId
21   });
22 }
```

Basket Provider and Client Hooks

`BasketProvider` keeps basket data in React context for the client. It receives the snapshot from the root loader, exposes lazy hydration, and updates through hooks.



initial render.

- `basket` : Full basket payload (optional, for eager client hydration).

Provider State Shape

The provider stores:

- `snapshot` : Cookie-safe basket summary.
- `current` : Full basket payload (when hydrated).
- `hydrated` : Whether hydration has been attempted.
- `error` : Hydration error info (if any).

The root loader intentionally passes only `basketSnapshot` to avoid eager basket creation.

```
1 // root.tsx (simplified)
2 const basketSnapshot = context.get(basketRe
3
4 const providers = [
5   // ...other providers
6   [BasketProvider, { snapshot: basketSnapsh
7 ] as const;
```

Lazy Hydration (useBasket)

`useBasket()` triggers a client-side fetch when:

- A snapshot exists (has `basketId`).
- No full basket is yet in context.

Note

The client-side fetch is useful for components, such as the minicart, that render outside a full page loader and still need full basket data.

```
1 import { useBasket } from "@providers/bask
2
3 export function CartBadge() {
4   const basket = useBasket();
5   return <span>{basket?.totalProductCount ?
6 }
```

Updating and Resetting the Basket

Use these hooks in client components after mutations or checkout completion. Common use cases include



inferred from route loaders because of the default lazy behavior.

```
1 import { useBasketUpdater, useBasketReset }
2
3 const setBasket = useBasketUpdater();
4 const resetBasket = useBasketReset();
5
6 // After a successful mutation
7 setBasket(updatedBasket);
8
9 // On checkout completion
10 resetBasket();
```

Advanced Configuration

Use these options only when needed.

1. **Eager hydration:** Set `mode: 'eager'` if a route always needs a full basket.
2. **Custom snapshot fields:** Add metadata (for example, pickup flags) via `calculateBasketSnapshot`.
3. **Force refresh:** Call `getBasket(context, { ensureBasket: true })` to hydrate server-side before action logic that requires a full basket.

Example custom snapshot:

```
1 createBasketMiddleware({
2   calculateBasketSnapshot: (basket) => ({
3     hasShipments: Boolean(basket.shipments?
4   }),
5 });
```

Basket Lifecycle Summary

Phase	Behavior	Example
Initial request	Middleware reads cookie → snapshot only	Cart badge shows count.
Full basket data	Loaded when the route or component explicitly needs it.	Cart page loader.

[Try for free](#)

	when the UI needs data.	component.
Updates and cleanup	Mutations update context/cookie. <code>destroyBasket()</code> clears state.	After checkout completion.

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

[Heroku](#)
[MuleSoft](#)
[Tableau](#)
[Commerce Cloud](#)
[Lightning Design System](#)
[Einstein](#)
[Quip](#)

POPULAR RESOURCES

[Documentation](#)
[Component Library](#)
[APIs](#)
[Trailhead](#)
[Sample Apps](#)
[AppExchange](#)

COMMUNITY

[Trailblazer Community](#)
[Events and Calendar](#)
[Partner Community](#)
[Blog](#)
[Salesforce Admins](#)
[Salesforce Architects](#)

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)

